PROGRAM 1

Create a table called Employee and execute the following Employee (EMPNo,ENAME,JOB,MANAGER_NO,SAL,COMMISSION)

a. Create a user and grant all permission to user.

```
-- Create a user and grant all permissions to the user--
CREATE USER 'myuser'@'localhost'
IDENTIFIED BY 'mypassword';
GRANT ALL PRIVILEGES ON p1.* TO 'myuser'@'localhost';
```

b. Insert any three records in the Employee table contains attributes (EMPNo,ENAME,JOB,MANAGER_NO,SAL,COMMISSION) and use rollback. Check the results.

```
create database p1;

use p1;

CREATE TABLE Employee (
   EMPNO INT,
   ENAME VARCHAR(50),
   JOB VARCHAR(50),
   MANAGER_NO INT,
   SAL DECIMAL(10, 2),
   COMMISSION DECIMAL(10, 2)
);

INSERT INTO Employee
VALUES
(1,'John_Doe','Manager',NULL,5000.00,1000.00),
(2,'Jane Smith','Developer',1,4000.00,500.00),
```

(3,'Bob Johnson','Analyst',1,3500.00,NULL);

ROLLBACK;

SELECT * FROM Employee;

EMPNO	ENAME	JOB	MANAGER_NO	SAL	COMMISSION
1	John_Doe	Manager	NULL	5000.00	1000.00
2	Jane Smith	Developer	1	4000.00	500.00
3	Bob Johnson	Analyst	1	3500.00	NULL

c. Add primary key constraints and not null constraint to the Employee table.

ALTER TABLE Employee

ADD PRIMARY KEY (EMPNO);

ALTER TABLE Employee

MODIFY ENAME VARCHAR(50) NOT NULL;

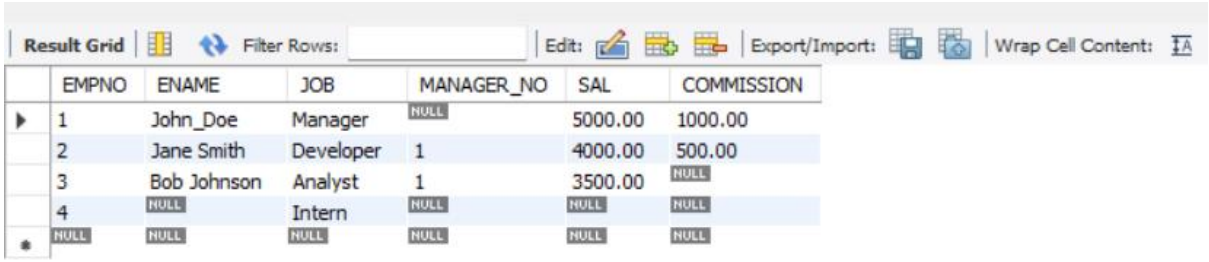
d. Insert null values to the Employee table and verify the result.

INSERT INTO Employee

VALUES

(5, NULL,'Intern', NULL, NULL, NULL);

SELECT * FROM Employee;



EMPNO	ENAME	JOB	MANAGER_NO	SAL	COMMISSION
1	John_Doe	Manager	NULL	5000.00	1000.00
2	Jane Smith	Developer	1	4000.00	500.00
3	Bob Johnson	Analyst	1	3500.00	NULL
4	NULL	Intern	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL

PROGRAM 2

Create a table called Employee that contain attributes EMPNO , ENAME, JOB, MGR, SAL and execute the following.

```
CREATE TABLE Employee1 (
    EMPNO INT,
    ENAME VARCHAR(50),
    JOB VARCHAR(50),
    MANAGER INT,
    SAL DECIMAL(10, 2)
);
```

a. Add a column commission with domain to the Employee table.

```
ALTER TABLE Employee1
ADD commission DECIMAL (10, 2);
```

b. Insert any five records into the table.

```
INSERT INTO Employee1 (EMPNO, ENAME, JOB, MGR, SAL, commission)
VALUES (101, 'John Doe', 'Manager', NULL, 5000.00, 1000.00);
INSERT INTO Employee1 (EMPNO, ENAME, JOB, MGR, SAL, commission)
VALUES (102, 'Jane Smith', 'Developer', 101, 4000.00, 500.00);
INSERT INTO Employee1 (EMPNO, ENAME, JOB, MGR, SAL, commission)
VALUES (103, 'Bob Johnson', 'Analyst', 101, 3500.00, NULL);
INSERT INTO Employee1 (EMPNO, ENAME, JOB, MGR, SAL, commission)
VALUES (104, 'Alice Jones', 'Designer', 101, 3800.00, 800.00);
INSERT INTO Employee1 (EMPNO, ENAME, JOB, MGR, SAL, commission)
VALUES (105, 'Michael Brown', 'Engineer', 101, 4200.00, 700.00);
```

SELECT * FROM Employee1;

EMPNO	ENAME	JOB	MANAGER	SAL	commission
101	John Doe	Manager	NULL	5000.00	1000.00
102	Jane Smith	Developer	101	4000.00	500.00
103	Bob Johnson	Analyst	101	3500.00	NULL
104	Alice Jones	Designer	101	3800.00	800.00
105	Michael Brown	Quality Analyst	101	4200.00	700.00

c. Update the column details of the job.

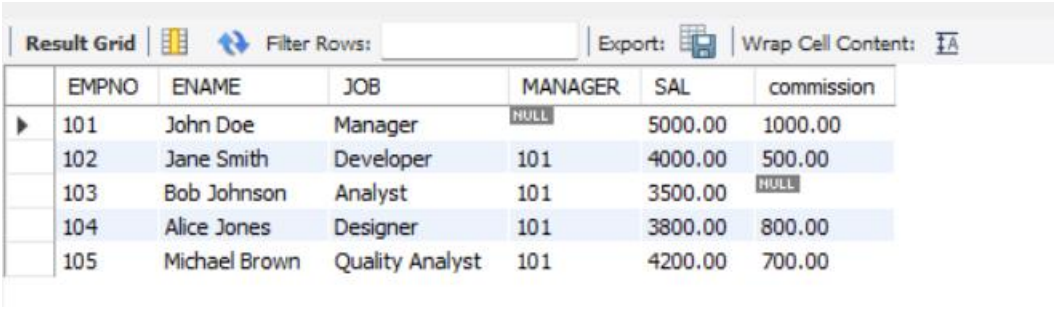
SET SQL_SAFE_UPDATES = 0;

UPDATE Employee1

SET JOB = 'Quality Analyst'

WHERE EMPNO = 105;

SELECT*FROM employee1



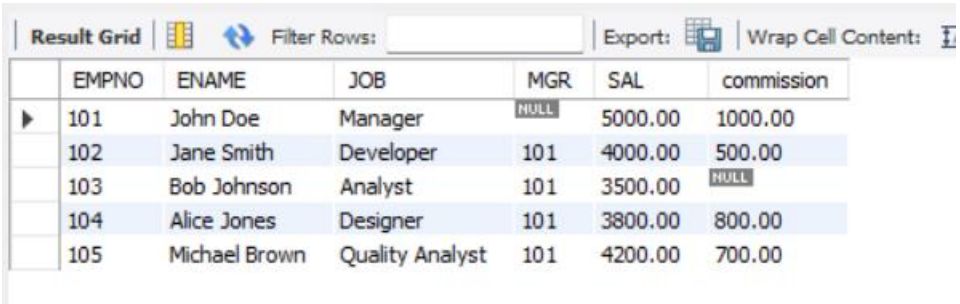
EMPNO	ENAME	JOB	MANAGER	SAL	commission
101	John Doe	Manager	NULL	5000.00	1000.00
102	Jane Smith	Developer	101	4000.00	500.00
103	Bob Johnson	Analyst	101	3500.00	NULL
104	Alice Jones	Designer	101	3800.00	800.00
105	Michael Brown	Quality Analyst	101	4200.00	700.00

d. Rename a column of the Employee table using the ALTER command.

ALTER TABLE Employee1

RENAME COLUMN MANAGER to MGR;

SELECT * FROM Employee1;



EMPNO	ENAME	JOB	MGR	SAL	commission
101	John Doe	Manager	NULL	5000.00	1000.00
102	Jane Smith	Developer	101	4000.00	500.00
103	Bob Johnson	Analyst	101	3500.00	NULL
104	Alice Jones	Designer	101	3800.00	800.00
105	Michael Brown	Quality Analyst	101	4200.00	700.00

e. Delete the Employee whose EMPNO is 105.

```
DELETE FROM Employee1

WHERE EMPNO = 105;

SELECT * FROM Employee1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

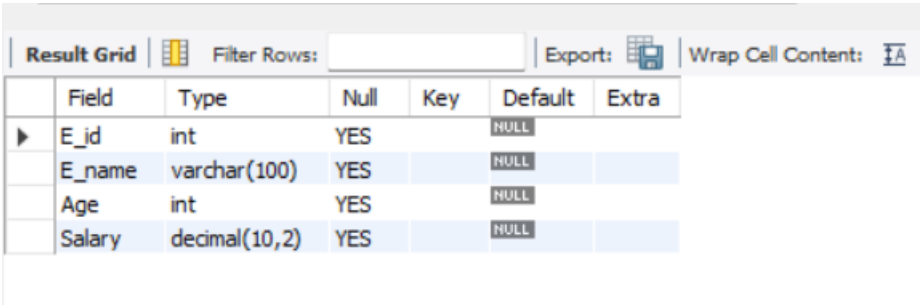
EMPNO	ENAME	JOB	MGR	SAL	commission
101	John Doe	Manager	NULL	5000.00	1000.00
102	Jane Smith	Developer	101	4000.00	500.00
103	Bob Johnson	Analyst	101	3500.00	NULL
104	Alice Jones	Designer	101	3800.00	800.00

PROGRAM 3

Queries using aggregates functions (COUNT, AVG, MIN, MAX, SUM) group by, order by Employee (E_id, E_name, Age, Salary)

a. Create Employee table containing all records E_id,E_name,Age,Salary.

```
create database employee2;

USE employee2;

CREATE TABLE Employee2 (E_id INT,

E_name VARCHAR(100),Age INT,

Salary DECIMAL(10, 2));

DESC Employee2;
```



Field	Type	Null	Key	Default	Extra
E_id	int	YES		NULL	
E_name	varchar(100)	YES		NULL	
Age	int	YES		NULL	
Salary	decimal(10,2)	YES		NULL	

```
INSERT INTO Employee2 (E_id, E_name, Age, Salary)

VALUES(101, 'John Doe', 30, 50000.00),

(102, 'ABC', 40, 60000.00),

(103, 'XYZ', 35, 45000.00),

(104, 'PQR', 36, 46000.00);

SELECT * FROM Employee2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

E_id	E_name	Age	Salary
101	John Doe	30	50000.00
102	ABC	40	60000.00
103	XYZ	35	45000.00
104	PQR	36	46000.00

b. Count number of Employee names Employee table.

SELECT COUNT(E_name) AS NumberOfEmployees FROM Employee2**;**

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

NumberOfEmployees
4

c. Find the Maximum age from Employee table.

SELECT MAX(Age) AS MaxAge FROM Employee2;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

MaxAge
40

d. Find the minimum age from Employee table.

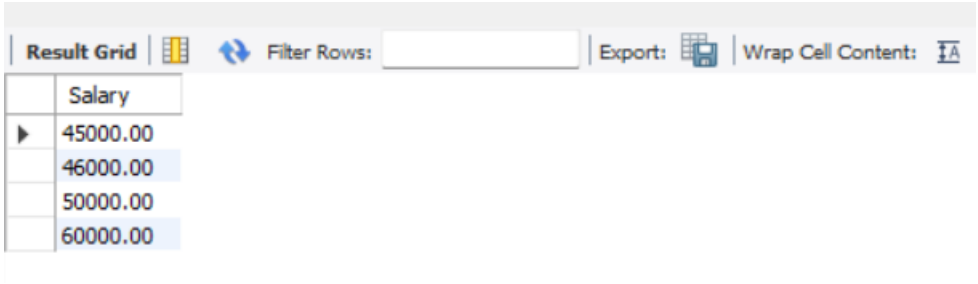
SELECT MIN(Age) AS MinAge FROM Employee2;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

MinAge
30

e. Find salaries of Employee in Ascending order.

SELECT Salary FROM Employee2 ORDER BY Salary ASC;

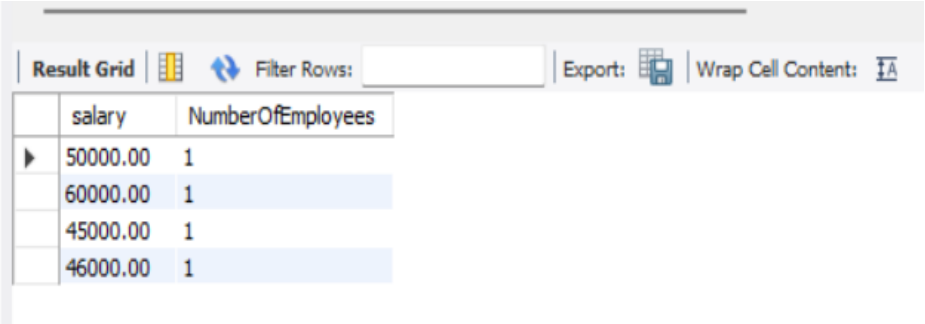

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Salary
45000.00
46000.00
50000.00
60000.00

f. Find the grouped salaries of employee.

SELECT salary ,

COUNT(*) AS NumberOfEmployees FROM employee2


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

salary	NumberOfEmployees
50000.00	1
60000.00	1
45000.00	1
46000.00	1

PROGRAM 4

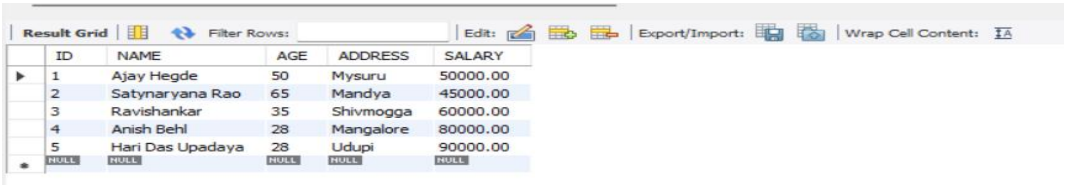
Create a row level triggered for the column table that would fire for INSERT and UPDATE or DELETE operations performed on CUSTOMERS table This trigger will display the salary difference between the old and new salary.

CUSTOMER(ID,NAME,AGE,ADDRESS,SALARY)

```
Create database CUSTOMERS;

USE CUSTOMERS;


-- CREATE CUSTOMER TABLE--

CREATE TABLE CUSTOMERS(

ID INT PRIMARY KEY,

NAME VARCHAR(255),

AGE INT,

ADDRESS VARCHAR(255),

SALARY DECIMAL(10, 2));


 -- TABLE INSERT STATEMENTS--

INSERT INTO CUSTOMERS VALUES (1, 'Ajay Hegde', 50, 'Mysuru', 50000.00);

INSERT INTO CUSTOMERS VALUES (2, 'Satynaryana Rao', 65, 'Mandya', 45000.00);

INSERT INTO CUSTOMERS VALUES (3, 'Ravishankar', 35, 'Shivmogga', 60000.00);

INSERT INTO CUSTOMERS VALUES (4, 'Anish Behl', 28, 'Mangalore', 80000.00);

INSERT INTO CUSTOMERS VALUES (5, 'Hari Das Upadaya', 28, 'Udupi', 90000.00);
```

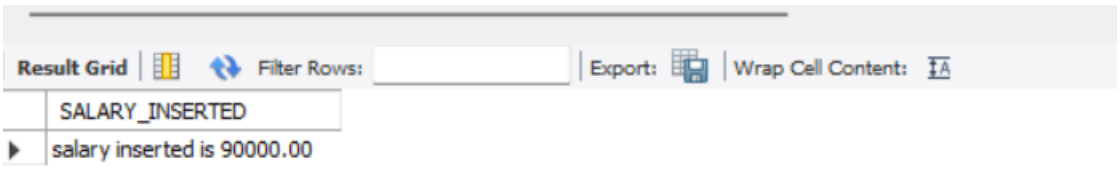
-- To Display Customer table--

```
SELECT * FROM CUSTOMERS;
```



ID	NAME	AGE	ADDRESS	SALARY
1	Ajay Hegde	50	Mysuru	50000.00
2	Satynaryana Rao	65	Mandya	45000.00
3	Ravishankar	35	Shivmogga	60000.00
4	Anish Behl	28	Mangalore	80000.00
5	Hari Das Upadaya	28	Udupi	90000.00
NULL	NULL	NULL	NULL	NULL

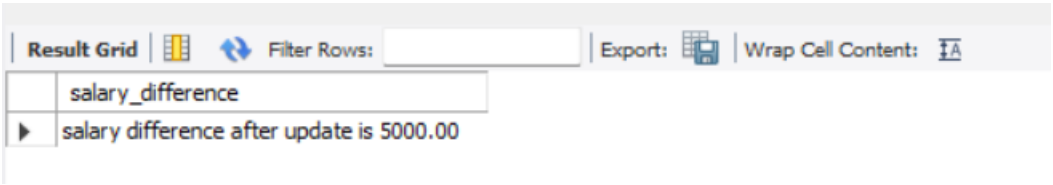
```
DELIMITER //

CREATE TRIGGER after_insert_salary_difference

AFTER INSERT ON CUSTOMERS

FOR EACH ROW

BEGIN

    SET @my_sal_diff = CONCAT('salary inserted is ', NEW.SALARY);

END;//

DELIMITER ;

-- TO INSERT VALUES AFTER TRIGGER

INSERT INTO CUSTOMERS

VALUES (6, 'amit kumar', 38, 'bijapur', 90000.00);

-- SELECT THE VARIABLE @my_sal_diff CREATED IN TRIGGER

SELECT @my_sal_diff AS SALARY_INSERTED;
```



SALARY_INSERTED
salary inserted is 90000.00

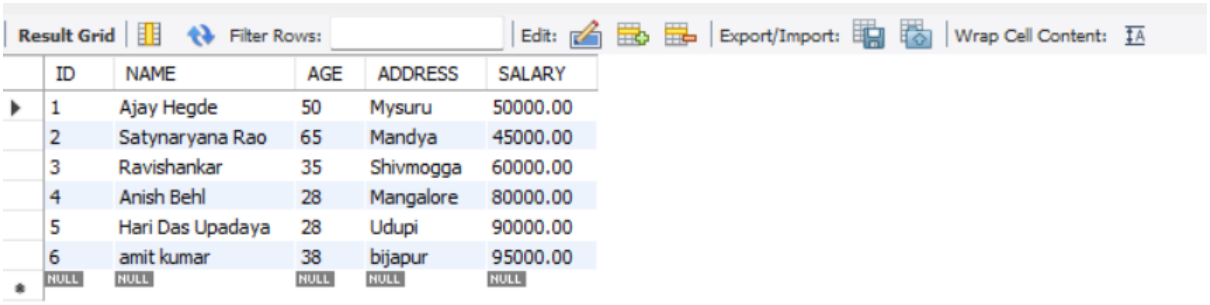
```
-- TO CREATE TRIGGER after_update_salary_difference

DELIMITER //

CREATE TRIGGER after_update_salary_difference
```

```
AFTER UPDATE ON CUSTOMERS

FOR EACH ROW

BEGIN

   DECLARE old_salary DECIMAL(10, 2);

   DECLARE new_salary DECIMAL(10, 2);

   SET old_salary = OLD.SALARY;

   SET new_salary = NEW.SALARY;

 SET @my_sal_diff = CONCAT('salary difference after update is ', NEW.SALARY - OLD.SALARY);

END;//

DELIMITER ;

 UPDATE CUSTOMERS SET SALARY=95000 WHERE ID=6;

 SELECT @my_sal_diff AS salary_difference;
```




```
-- DELETE TRIGGER

DELIMITER //

CREATE TRIGGER after_delete_salary_difference

AFTER DELETE ON CUSTOMERS

FOR EACH ROW

BEGIN

   SET @my_sal_diff = CONCAT('salary deleted is ', OLD.SALARY);

END;//

DELIMITER ;
```

SELECT * FROM CUSTOMERS;

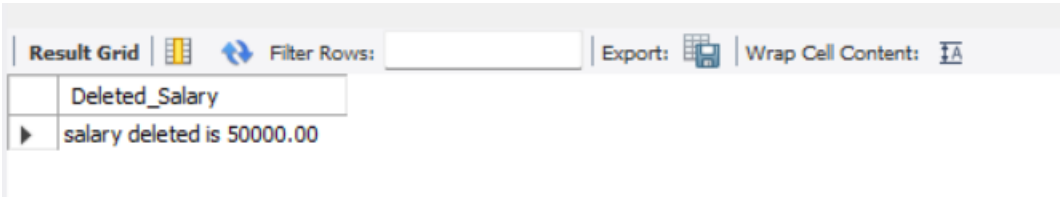


Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ID	NAME	AGE	ADDRESS	SALARY
1	Ajay Hegde	50	Mysuru	50000.00
2	Satynaryana Rao	65	Mandya	45000.00
3	Ravishankar	35	Shivmogga	60000.00
4	Anish Behl	28	Mangalore	80000.00
5	Hari Das Upadaya	28	Udupi	90000.00
6	amit kumar	38	bijapur	95000.00
NULL	NULL	NULL	NULL	NULL

DELETE FROM CUSTOMERS WHERE ID=1;

SELECT @my_sal_diff AS Deleted_Salary;



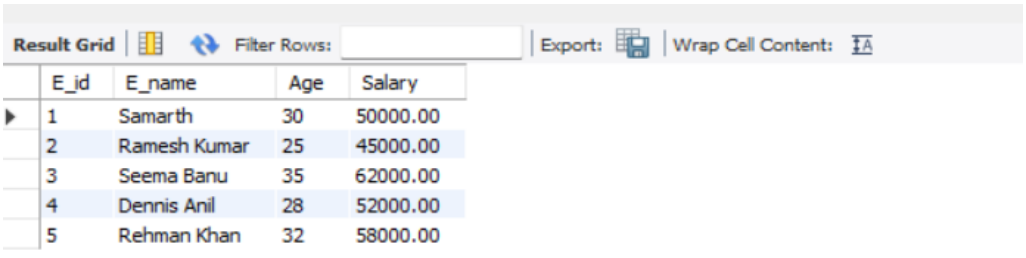
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Deleted_Salary
salary deleted is 50000.00

PROGRAM 5

Create cursor for Employee table and extract the values from the table. Declare the variables open the cursor and extract the values from the cursor. Close cursor Employee(E_id,E_name,Age,Salary).

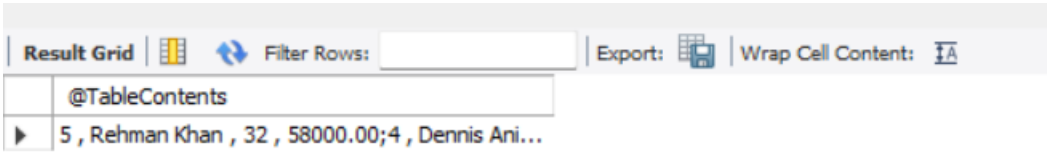
```
CREATE DATABASE pg;

USE pg;

CREATE TABLE Employee1 (
    E_id INT,
    E_name VARCHAR(255),
    Age INT,
    Salary DECIMAL(10, 2)
);

INSERT INTO Employee1 (E_id, E_name, Age, Salary) VALUES (1, 'Samarth', 30, 50000.00);

INSERT INTO Employee1 (E_id, E_name, Age, Salary) VALUES (2, 'Ramesh Kumar', 25, 45000.00);

INSERT INTO Employee1 (E_id, E_name, Age, Salary) VALUES (3, 'Seema Banu', 35, 62000.00);

INSERT INTO Employee1 (E_id, E_name, Age, Salary) VALUES (4, 'Dennis Anil', 28, 52000.00);

INSERT INTO Employee1 (E_id, E_name, Age, Salary) VALUES (5, 'Rehman Khan', 32, 58000.00);

SELECT * FROM Employee1;
```



E_id	E_name	Age	Salary
1	Samarth	30	50000.00
2	Ramesh Kumar	25	45000.00
3	Seema Banu	35	62000.00
4	Dennis Anil	28	52000.00
5	Rehman Khan	32	58000.00

```
DELIMITER $$
CREATE PROCEDURE gettable1(INOUT TableContents VARCHAR(4000))
BEGIN
   DECLARE finished INTEGER DEFAULT 0;
   DECLARE content VARCHAR(100) DEFAULT '';
   DECLARE curName1 CURSOR FOR
      SELECT CONCAT(E_id, ' , ', E_name, ' , ', Age, ' , ', Salary)
      FROM Employee1
      ORDER BY E_id DESC;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = 1;
 OPEN curName1;
getName1: LOOP
      FETCH curName1 INTO content;
 IF finished = 1 THEN
          LEAVE getName1;
      END IF;
SET TableContents = CONCAT(IFNULL(TableContents,''), content, ';');
   END LOOP getName1;
 CLOSE curName1;
END$$
DELIMITER ;

SET @TableContents = '';
CALL gettable1(@TableContents);
```

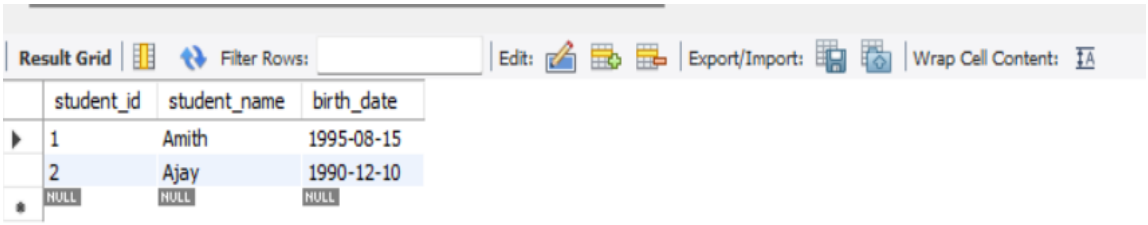
SELECT @TableContents;



PROGRAM 6

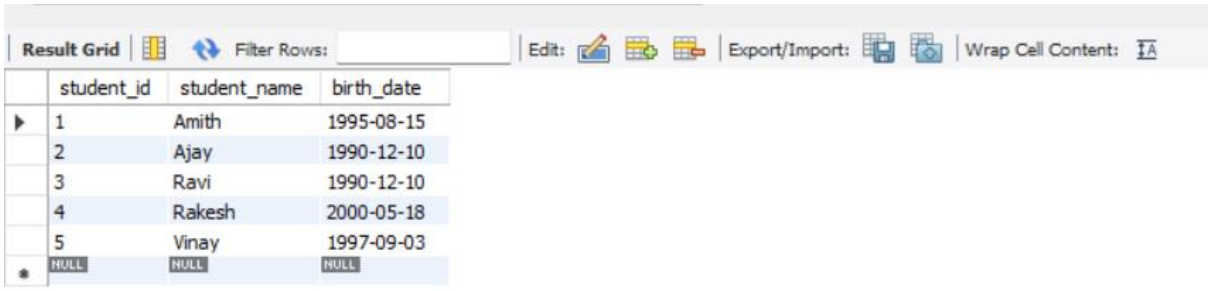
Write a PL/SQL block of code using parameterized cursor that will merge the data available in the newly created table N_RollCall with the data available in the table D_RollCall. If the data in the first table already exist.

```
CREATE DATABASE ROLLCALL;

USE ROLLCALL;

-- Create N_RollCall table--

CREATE TABLE N_RollCall (

student_id INT PRIMARY KEY,

student_name VARCHAR(255),

birth_date DATE);

 -- Create O_RollCall table with common data--

CREATE TABLE O_RollCall (

student_id INT PRIMARY KEY,

student_name VARCHAR(255),

birth_date DATE);

 INSERT INTO O_RollCall (student_id, student_name, birth_date)

VALUES (1, 'Amith', '1995-08-15');

INSERT INTO O_RollCall (student_id, student_name, birth_date)

VALUES (2, 'Ajay', '1990-12-10');

 SELECT * FROM O_RollCall;
```



student_id	student_name	birth_date
1	Amith	1995-08-15
2	Ajay	1990-12-10
NULL	NULL	NULL

```
INSERT INTO N_RollCall (student_id, student_name, birth_date)
VALUES (1, 'Amith', '1995-08-15');
INSERT INTO N_RollCall (student_id, student_name, birth_date)
VALUES (2, 'Ajay', '1990-12-10');
INSERT INTO N_RollCall (student_id, student_name, birth_date)
VALUES (3, 'Ravi', '1990-12-10');
INSERT INTO N_RollCall (student_id, student_name, birth_date)
 VALUES (4, 'Rakesh', '2000-05-18');
INSERT INTO N_RollCall (student_id, student_name, birth_date)
VALUES (5, 'Vinay', '1997-09-03');
SELECT * FROM N_RollCall;
```

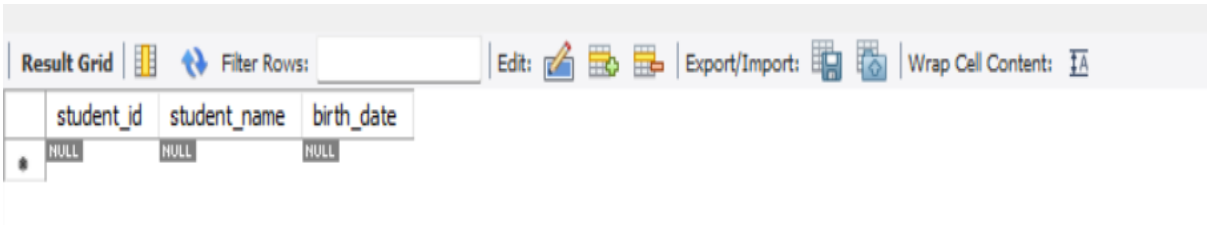

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	student_name	birth_date
1	Amith	1995-08-15
2	Ajay	1990-12-10
3	Ravi	1990-12-10
4	Rakesh	2000-05-18
5	Vinay	1997-09-03
NULL	NULL	NULL

```
DELIMITER //
CREATE PROCEDURE merge_rollcall_data()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE n_id INT;
    DECLARE n_name VARCHAR(255);
    DECLARE n_birth_date DATE;
    -- Declare cursor for N_RollCall table
    DECLARE n_cursor CURSOR FOR
```

```
    SELECT student_id, student_name, birth_date FROM N_RollCall;
  -- Declare handler for cursor
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
  -- Open the cursor
  OPEN n_cursor;
  -- Start looping through cursor results
  cursor_loop: LOOP
    -- Fetch data from cursor into variables
    FETCH n_cursor INTO n_id, n_name, n_birth_date;
    -- Check if no more rows to fetch
    IF done THEN LEAVE cursor_loop;
    END IF;
    -- Check if the data already exists in O_RollCall
    IF NOT EXISTS (SELECT 1 FROM O_RollCall WHERE student_id = n_id) THEN
      -- Insert the record into O_RollCall
      INSERT INTO O_RollCall(student_id, student_name, birth_date)
      VALUES (n_id, n_name, n_birth_date);
    END IF;
  END LOOP;
  -- Close the cursor
  CLOSE n_cursor;
END//
DELIMITER ;
```

```
CALL merge_rollcall_data();

DELETE FROM O_RollCall;

SELECT * FROM O_RollCall;
```



PROGRAM 7

Install an open source NOSQL database MongoDB and perform basic CRUD(Create, Read, Update and Delete)operations. Execute MongoDB basic queries using CRUD operations.

Create Database CollegeDB and collection name Students.

a. CREATE

> db.students.insertOne({name:"Rahul",age:21,course:"BCA"})

```
> db.students.insertOne({name:"Rahul",age:21,course:"BCA"})
< {
    acknowledged: true,
    insertedId: ObjectId('6a1b2ca3b40376ca469c4eb3')
  }
```

> db.students.insertMany([{name:"Mahesh",age:23,course:"CSE"},{name:"spoorthi",age:21,

course:"AIML"},{name:"Pavi",age:22,course:"DATASCIENCE"}])

```
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6a1b2dbcb40376ca469c4eb5'),
      '1': ObjectId('6a1b2dbcb40376ca469c4eb6'),
      '2': ObjectId('6a1b2dbcb40376ca469c4eb7')
    }
  }
```

b. READ

> db. students.find()

```
> db.students.find()
< {
    _id: ObjectId('6a1b2ca3b40376ca469c4eb3'),
    name: 'Rahul',
    age: 21,
    course: 'BCA'
  }
```

```
{
  _id: ObjectId('6a1b2d48b40376ca469c4eb4'),
  name: 'Mahesh',
  age: 23,
  course: 'CSE'
}
{
  _id: ObjectId('6a1b2dbcb40376ca469c4eb5'),
  name: 'Mahesh',
  age: 23,
  course: 'CSE'
}
{
  _id: ObjectId('6a1b2dbcb40376ca469c4eb6'),
  name: 'spoorthi',
  age: 21,
  course: 'AIML'
}
{
  _id: ObjectId('6a1b2dbcb40376ca469c4eb7'),
  name: 'Pavi',
  age: 22,
  course: 'DATASCIENCE'
}
```

c. UPDATE

> db.students.updateOne({name:"Pavi"},{$set:{age:22}})

```
> db.students.updateOne({name:"Pavi"},{$set:{age:22}})
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 0,
    upsertedCount: 0
  }
```

d. DELETE

> db.students.deleteOne({name:"spoorthi"})

```
> db.students.deleteOne({name:"spoorthi"})
< {
    acknowledged: true,
    deletedCount: 1
  }
```

Verify

> db.students.find()

```
> db.students.find()
< {
    _id: ObjectId('6a1b2ca3b40376ca469c4eb3'),
    name: 'Rahul',
    age: 21,
    course: 'BCA'
  }
  {
    _id: ObjectId('6a1b2d48b40376ca469c4eb4'),
    name: 'Mahesh',
    age: 23,
    course: 'CSE'
  }
  {
    _id: ObjectId('6a1b2dbcb40376ca469c4eb5'),
    name: 'Mahesh',
    age: 23,
    course: 'CSE'
  }
  {
    _id: ObjectId('6a1b2dbcb40376ca469c4eb7'),
    name: 'Pavi',
    age: 22,
    course: 'DATASCIENCE'
  }
```